

Cálculos Caminhos Mínimos com GPU no CUDA

Trabalho 2 - Programação Paralela

Maryana Catonho

• Introdução

O relatório apresenta o projeto para calcular o problema da caminhos mínimos, comparando o desempenho com com uma versão sequencial com CPU com outra versão paralela com GPU utilizando o CUDA.

O grafo é representado por uma matriz de dimensão $N \times N$, onde N é o número de vértices; $M[i][j]$ armazenam o peso da aresta de i para j . Caso não tenha o valor, a aresta é representada por INF (implementado como 999999 para ser um valor grande).

Os grafos utilizados para o teste vão de 1024, 2048, 3072 a 4096 com densidades de arestas de 25%, 50% e 75%.

• Implementação

O algoritmo escolhido para resolver o problema foi o Floyd-Warshall, pois utiliza recorrência para encontrar o caminho mínimo de i a j passando por vértices intermediários k onde $0 \leq k < N$. O algoritmo possui complexidade $O(N^3)$, o que é muito pesado em grafos grandes, o que mostra a necessidade de uma implementação com paralelismo.

A implementação do sequencial usa três loops aninhados, com o loop externo itera sobre os vértices intermediários; também dois loops internos que iteram sobre todos os pares de origem destino.

Para o mapeamento de threads e blocos, em cada iteração do loop, a atualização da matriz de distancias envolve o cálculo de $N \times N$ elementos, tornando o cálculo de cada independente dos outros. O índice i é mapeado para o *blockIdx.y* e *threadIdx.y*, e o j é mapeado para o *blockIdx.x* e *threadIdx.x*.

A principal matriz de distancias (dist) foi alocada e acessada na memória global da GPU, ela é lida e escrita por todas as threads, mesmo sendo demorado, permite que todas as threads acessem todos os dados.

Os códigos foram rodados com uma NVIDIA GeForce GTX 1650 com memoria global de 4095 MB. Também conhecido como: meu computador pessoal.

• Resultados

Os resultados foram muito satisfatórios

PS: as variações no tempo sequencial nas diferentes densidades são por causa das flutuações do sistema, provavelmente não por conta do algoritmo

<i>Densidade 25%</i>	CPU	CUDA	speedup
1024	4.614s	0.319s	14.46x
2048	37.914s	1.007s	37.65x
3072	128.230s	2.395s	53.54x
4096	304.173s	4.835s	62.91x

TABELA 1 – densidade 25% CPU x CUDA

<i>Densidade 50%</i>	CPU	CUDA	speedup
1024	4.985s	0.264s	18.88x
2048	37.249s	1.045s	35.64x
3072	128.657s	2.408s	53.42x
4096	302.063s	4.830s	62.53x

TABELA 2 – densidade 50% CPU x CUDA

<i>Densidade 75%</i>	CPU	CUDA	speedup
1024	5.248s	0.192s	27.33x
2048	38.301s	1.045s	36.65x
3072	123.520s	2.442s	50.58x
4096	301.775s	4.773s	63.22x

TABELA 3 – densidade 75% CPU x CUDA

	CPU	CUDA	speedup
1024	4.948s	0.258s	19.12x
2048	37.821s	1.032s	36.62x
3072	126.802s	2.415s	52.50x
4096	302.670s	4.813s	62.89x

TABELA 4 - média de todas as densidades CPU x CUDA

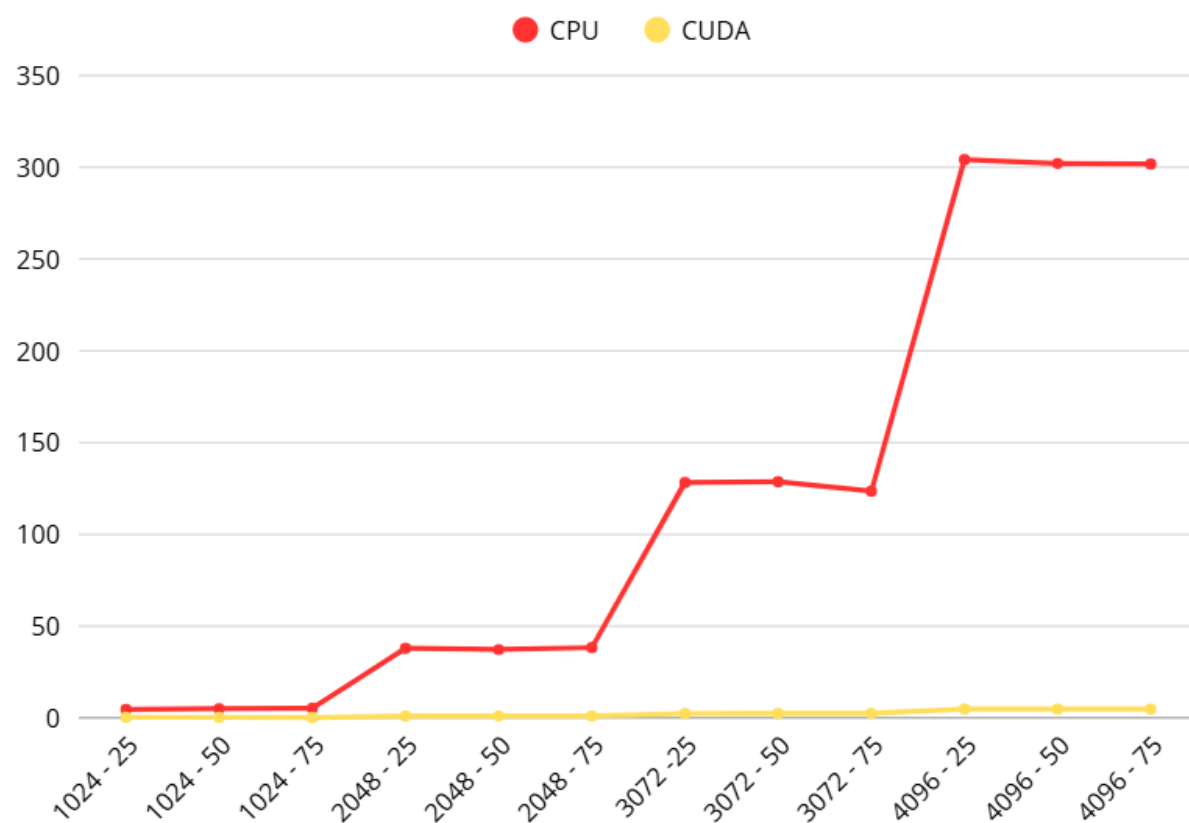


GRÁFICO 1 - comparação sequencial X CUDA em todos

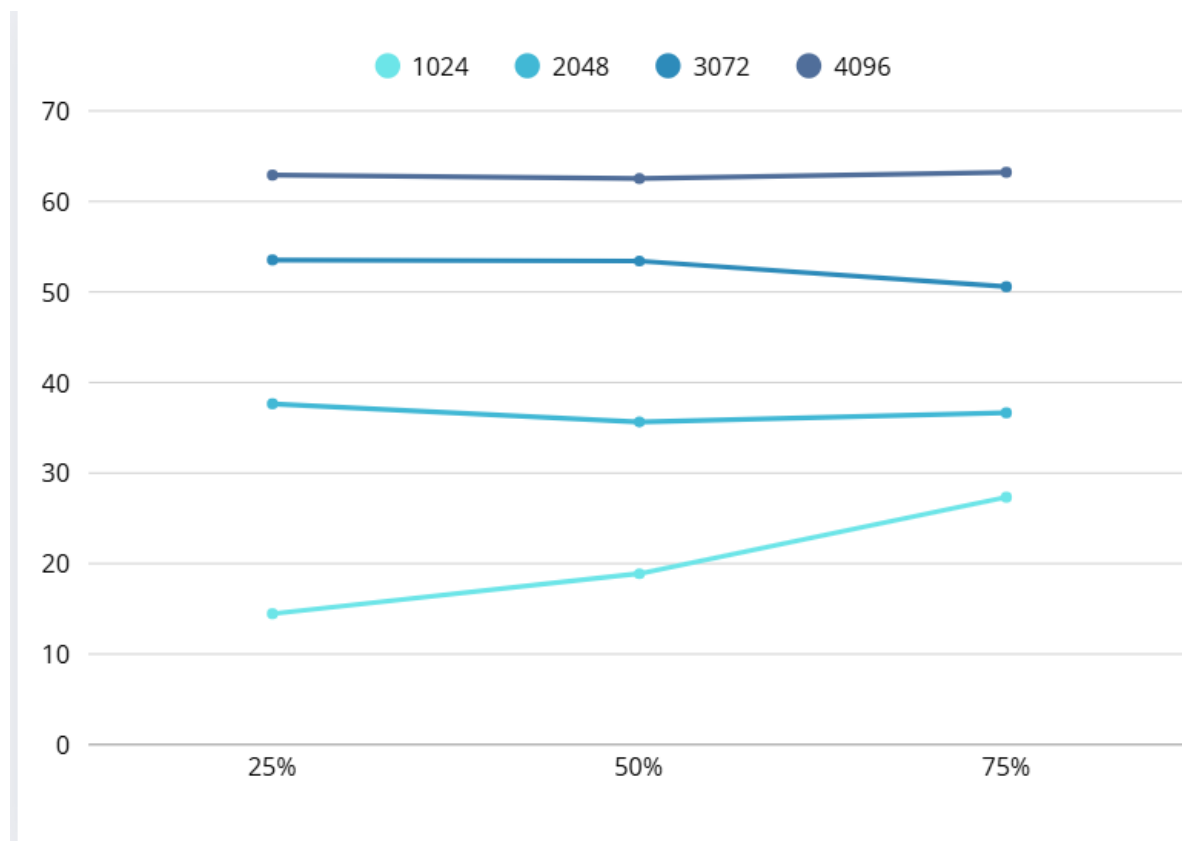


GRÁFICO 2 – speedups

• Análise

Como esperado, os resultados com o CUDA foram muito mais rápidos que os sequenciais.

No sequencial, de um pro outro houve um aumento de 8x, o que comprova a complexidade $O(N^3)$; No CUDA, também há um aumento de execução, mas não tão grande quanto o outro caso, sendo de 4x entre o 1024 e 4096 (primeiro e último exemplo) e não entre um e outro.

O speedup comprova a eficiência, indo de 19.12x em 1024 e até 62.89x em 4096. O speedup aumenta com o tamanho do problema, demonstrando escalabilidade do paralelismo.

Entretanto, há algumas limitações, como por exemplo, a dependência no loop externo k por ele não poder ser paralelizado; também o acesso não-sequencial na memória global.

• Conclusão

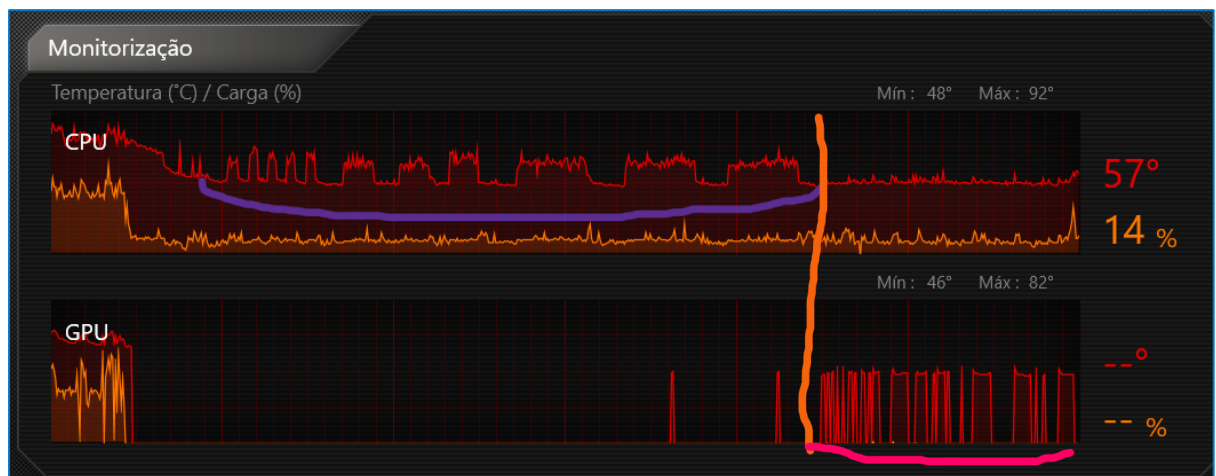
A implementação com CUDA foi mais eficiente que a sequencial com uma diferença extremamente notável para o problema de caminhos mínimos com o algoritmo Floyd-Warshall. Foi concluído que, a CPU com processamento paralelo domina a sobrecarga de transferência de dados na medida que o grafo aumenta.

Assim, utilizar CUDA mostrou ser muito mais eficaz na aceleração de problemas de grafos grandes e complexos.

Futuras implementações e otimizações incluem utilizar melhor uma memória compartilhada, carregando as linhas e colunas de forma compartilhada e garantindo acessos mais rápidos dentro do bloco.

• Bônus curiosidade

Enquanto rodava o código, foi possível ver a CPU e GPU sendo utilizadas na aplicação de monitoramento do computador!!



Em roxo, é o momento rodando os sequenciais, os menores do começo são os exemplos menores que deram um resultado de poucos segundos; em laranja é o momento que começo o de CUDA; em rosa, é utilizando a GPU. Dá pra observar que, quando eu estava rodando em um, o outro não estava ativo, e quando comecei a GPU, a o uso da CPU estabilizou!